Stratum Protocol

Pool Connection Explained in Simple Words

How Your PDSA-FPGA Miner Talks to a Mining Pool

Generated: 2026-06-20 12:08

NOTE: The Stratum/pool code has been removed from the current project. This PDF is kept as reference material.

Table of Contents

1 Big Picture: Your Miner and the Pool
2 What is a Mining Pool?
3 What is Stratum?
4 The Conversation: Step by Step
5 Step 1: Connect (TCP Handshake)
6 Step 2: Subscribe ("Give me work")
7 Step 3: Authorize ("This is me")
8 Step 4: Receive Job (New Block to Mine)
9 Step 5: Parse the Job into a Block Header
10 Step 6: Feed to PL and Start Mining
11 Step 7: Submit Share ("I found one!")
12 Step 8: Repeat (Keep Mining)
13 Real JSON Messages (Copy-Paste Examples)
14 Where in the Code?
15 What You Need to Change for Your Pool

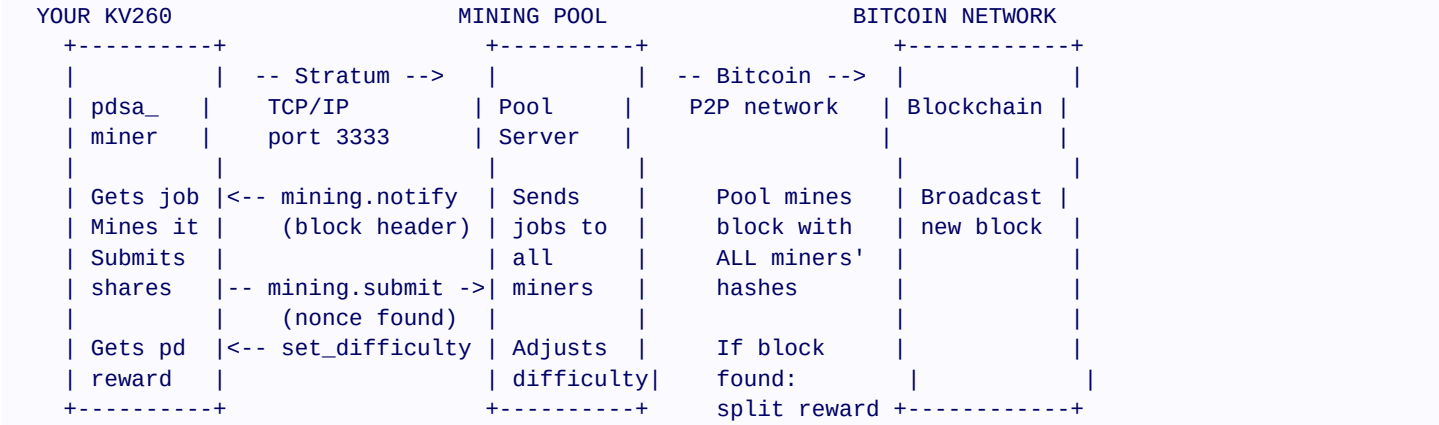
1. Big Picture: Your Miner and the Pool

Imagine you are alone trying to win the Bitcoin lottery. You buy lottery tickets (compute hashes) but the odds are terrible - you might try for years and never win a block.

Now imagine you join a lottery syndicate (a mining pool). Everyone in the syndicate pools their tickets together. When ANY member wins, the prize is split among everyone based on how many tickets they contributed. You get smaller but REGULAR payouts instead of hoping for one huge win.

That is EXACTLY what a mining pool does. Your FPGA miner sends its work (shares) to the pool. The pool combines work from thousands of miners. When the pool finds a Bitcoin block, everyone gets paid proportionally.

Big Picture Diagram



NOTE: Your FPGA does NOT connect directly to the Bitcoin network. It connects to a POOL, and the pool handles the Bitcoin network. This is how ALL modern miners work - solo mining is virtually impossible.

2. What is a Mining Pool?

A mining pool is a server that coordinates many miners to work together. Today, almost 100% of all Bitcoin mining happens through pools.

Why Join a Pool?

- Solo mining: You need ~200 TH/s (terahashes/sec) to find even 1 block per year. Your FPGA does 125 MH/s - that's 1.6 million times slower. You would NEVER find a block alone.
- Pool mining: You submit SHARES (hashes that almost meet the target, but not quite). The pool counts your shares and pays you proportionally. You get daily payouts in small amounts.
- Example: If the pool finds 10 blocks per day and you contributed 0.01% of the pool's hashrate, you get 0.01% of 6.25 BTC = 0.000625 BTC per day (~$15-20 at current prices).

Famous Pools (any will work with our miner)

- F2Pool (btc.f2pool.com:3333) - One of the largest, supports Bitcoin and Kaspa.
- ViaBTC (viabtc.com:3333) - Large Chinese pool.
- Poolin (poolin.com:3333) - Multi-coin pool.
- CKPool (solo.ckpool.org:3333) - Solo mining pool (if you want to try luck).

NOTE: You need to create a FREE account on the pool's website and get your worker credentials. Usually just a wallet address + optional worker name.

3. What is Stratum?

Stratum is the protocol (language) that miners and pools use to talk to each other. It was invented in 2012 and replaced the older getwork protocol. Stratum v1 is the most widely used version today.

Key Facts About Stratum

- It uses TCP (same protocol as web browsing, email). Your miner opens a TCP connection to the pool's IP address on port 3333.
- Messages are in JSON format - text-based and human-readable. You can read them with your eyes!
- The pool SENDS work to the miner (push-based, not poll-based). The miner doesn't ask for work - the pool sends it automatically.
- A session typically lasts hours or days. The miner stays connected continuously.
- If disconnected, the miner reconnects automatically (our code does this every 30 seconds).

Stratum vs Regular Internet

```
Regular Web Browsing:
  Your PC -- "Give me google.com" --> Google server
  Google server -- "Here's the page" --> Your PC
  (You ask first, server responds)

  Stratum Mining:
  Your Miner -- "I'm here, give me work" --> Pool
  Pool -- "OK, here's a block to mine" --> Your Miner
  Pool -- "Here's a NEWER block" --> Your Miner (30 sec later)
  Pool -- "Here's a NEWER block" --> Your Miner (30 sec later)
  ...
  Your Miner -- "I found a nonce!" --> Pool
  (Pool sends work automatically, miner submits when found)
```

stratum_client.c in the original code

The original Stratum client (420 lines of C, now removed) implemented all the necessary messages. It connected, subscribed, authorized, received jobs, and submitted shares using standard Linux sockets (socket(), connect(), send(), recv()). The current project runs in standalone mode with no pool dependency.

4. The Conversation: Step by Step

Here is the COMPLETE conversation between your miner and the pool, from start to continuous mining:

```
MINER> [Opens TCP connection to pool.bitcoin.com:3333]
  POOL>  [TCP connection established]

  MINER> mining.subscribe ("PDSA-FPGA/1.0")
  POOL>  Result: subscription_id, extranonce1, extranonce2_size

  MINER> mining.authorize ("worker1", "x")
  POOL>  Result: true (authorized)

  ====== MINING BEGINS ======

  POOL>  mining.notify (job_id=1, version, prev_hash,
         merkle_root, nbits, ntime, clean_jobs=true)
  MINER> [Parses job, computes midstate, sends to PL, starts hashing]

  ... 30 seconds pass ...

  POOL>  mining.notify (job_id=2, version, prev_hash,
         merkle_root, nbits, ntime, clean_jobs=true)
  MINER> [Switches to new job, abandons old one]

  ... miner finds a nonce ...

  MINER> mining.submit (worker1, job_id=2, ntime, nonce)
  POOL>  Result: true (share accepted!)

  POOL>  mining.notify (job_id=3, ...)
  MINER> [Gets new job, keeps mining]

  ... this continues forever ...
```

The miner NEVER sends a message asking for work. The pool PUSHES new jobs whenever there's a new block (approximately every 10 minutes, but pools send new jobs every 30-60 seconds to update transactions).

5. Step 1: Connect (TCP Handshake)

The first thing your miner does is open a TCP connection to the pool.

What is TCP?

TCP (Transmission Control Protocol) is the same technology that your web browser uses to load websites. It creates a reliable, two-way communication channel between two computers. Think of it like a phone call - both sides can talk, and they know when the other is listening.

In Code (stratum_client.c)

```
int stratum_connect(stratum_ctx_t *ctx,
                    const char *host, int port) {
      // Step 1: Look up the pool's IP address
      struct hostent *server = gethostbyname(host);
      // "pool.bitcoin.com" -> "192.168.1.100" (or whatever)

      // Step 2: Create a socket (like getting a phone)
      ctx->sock = socket(AF_INET, SOCK_STREAM, 0);

      // Step 3: Set the pool's address
      server_addr.sin_family = AF_INET;
      server_addr.sin_port = htons(port); // 3333
      server_addr.sin_addr = *server->h_addr;

      // Step 4: Connect (like dialing the phone)
      connect(ctx->sock, &server_addr, sizeof(server_addr));
      // If this fails: pool is down, no internet, etc.

      return 0; // Connected!
  }
```

What Can Go Wrong?

- No internet: connect() returns -1. Our code retries every 30 seconds.
- Wrong hostname: gethostbyname() fails. Check POOL_HOST spelling.
- Firewall: Port 3333 must be allowed OUTBOUND (not inbound). Most home routers allow this by default.
- Pool down: connect() times out after ~10 seconds. Try a different pool.

NOTE: Your KV260 needs an Ethernet cable connected to your router. The router provides internet access via DHCP (automatic IP address). No special router configuration needed - standard home internet works.

6. Step 2: Subscribe ("Give me work")

Once connected, the miner immediately sends a subscribe message. This tells the pool: "I'm a miner, please add me to your list and send me work."

The Message

```
MINER sends: {"id":1,"method":"mining.subscribe",
                "params":["PDSA-FPGA/1.0"]}
```

Broken down:

- id: 1 - A counter so we can match responses to requests.
- method: "mining.subscribe" - The command (Stratum built-in).
- params: ["PDSA-FPGA/1.0"] - Our miner name and version (the pool logs this).

The Pool's Response

```
POOL responds: {"id":1,"result":[
      [["mining.notify","ae6812eb4cd9"],
       ["mining.set_difficulty","1"]],
      "08000002",       <- extranonce1 (unique per connection)
      4                  <- extranonce2_size (bytes)
  ], "error":null}
```

The important parts are:

- extranonce1: A hex string unique to this connection. The miner includes this in every share to identify itself.
- extranonce2_size: Number of bytes the miner can use for extra nonce (usually 4).
- The subscriptions: mining.notify (new jobs) and mining.set_difficulty (target adjustment).

Why This Matters

Without subscribing, the pool doesn't know you exist and won't send you any work. This is like registering at a hotel front desk before getting a room key.

7. Step 3: Authorize ("This is me")

Next, the miner tells the pool who it is - specifically, which Bitcoin address should get the mining rewards.

The Message

```
MINER sends: {"id":2,"method":"mining.authorize",
              "params":["worker1","x"]}
```

Broken down:

- params[0]: "worker1" - This is YOUR worker name. Usually: YourBitcoinAddress.WorkerName. For example: "bc1qabc123def456.rig1"
- params[1]: "x" - Password. Most pools accept "x" as default. Some pools have a specific password.

The Pool's Response

```
POOL responds: {"id":2,"result":true,"error":null}
```

If result is TRUE, you're authorized and mining can begin. If FALSE, check your worker name and password.

Worker Name Format by Pool

```
F2Pool:   YourBitcoinAddress.WorkerName
            Example: bc1qabc123def456.rig1

  ViaBTC:   YourBitcoinAddress.WorkerName
            Example: bc1qabc123def456.rig1

  CKPool:   YourBitcoinAddress
            Example: bc1qabc123def456
```

NOTE: You MUST create your worker on the pool's website FIRST. Go to the pool's website, sign up, create a worker, and use that worker name here. The wallet address you provide is where your earnings will be sent.

8. Step 4: Receive Job (New Block to Mine)

Once authorized, the pool immediately sends a mining.notify message - THIS IS THE ACTUAL MINING JOB. This is the most important message.

The Message

```
POOL sends: {"method":"mining.notify",
    "params":["1",                    <- job_id
              "0000000000000000...",   <- prev_hash (32 bytes hex)
              "0100000000000000...",   <- coinbase1 (first part)
              "ffffffff...",           <- coinbase2 (second part)
              ["merkle1","merkle2",...],<- merkle_branch (list of hashes)
              "20000000",              <- version
              "1c2ac4af",              <- nbits (difficulty target)
              "61cfa8e4",              <- ntime (timestamp)
              true                     <- clean_jobs (abandon old jobs?)
            ]
  }
```

Broken down simply:

- job_id: A unique ID for this job. We submit shares with this ID.
- prev_hash: The hash of the previous block (forms the blockchain).
- coinbase1 + coinbase2: The coinbase transaction (where the reward goes).
- merkle_branch: A list of hashes needed to compute the Merkle root.
- version: Block version number (usually 0x20000000 for version 2 blocks).
- nbits: The difficulty target in compressed format (4 bytes).
- ntime: Current timestamp (4 bytes, Unix epoch).
- clean_jobs: If TRUE, abandon ALL old jobs immediately and only mine this one.

What Your Miner Does With This

Your miner (in block_header_parser.c) takes these fields and builds the 80-byte block header:

```
Bitcoin Block Header (80 bytes):
  +----------------+--------+
  | Field          | Bytes  |
  +----------------+--------+
  | Version        |  4     |
  | Previous Hash  | 32     |
  | Merkle Root    | 32     |
  | Timestamp      |  4     |
  | Bits (target)  |  4     |
  | Nonce          |  4     |  <-- YOUR FPGA ITERATES THIS
  +----------------+--------+
```

The Merkle root is computed from coinbase1 + merkle_branch + coinbase2. The midstate is SHA-256 of the first 512 bits of this header. The FPGA iterates the nonce field.

9. Step 5: Parse the Job into a Block Header

The block_header_parser.c module takes the Stratum job notification and converts it into the 80-byte block header that your FPGA can understand.

The Math Behind the Merkle Root

The Merkle root is computed by double-SHA-256 hashing pairs of hashes together in a tree. The coinbase transaction is at the bottom, and you combine it with the merkle_branch hashes from the pool:

```
// Step 1: Start with coinbase hash
  hash = SHA256d(coinbase1 + extranonce1 + extranonce2 + coinbase2)

  // Step 2: Combine with each merkle branch hash
  for each merkle_hash in merkle_branch:
      hash = SHA256d(hash + merkle_hash)

  // Step 3: The result is the Merkle root
  merkle_root = hash

  // Step 4: Build the 80-byte header
  header[0..3]    = version (little-endian)
  header[4..35]   = prev_hash
  header[36..67]  = merkle_root
  header[68..71]  = ntime
  header[72..75]  = nbits
  header[76..79]  = nonce (0 initially, FPGA changes this)
```

Computing the Midstate

The 80-byte header is processed by SHA-256 in two 512-bit blocks:

```
Block 1 (64 bytes): header[0..63] = first 64 bytes of 80-byte header
  Block 2 (64 bytes): header[64..79] + padding + length

  Midstate = SHA-256_compress(IV, Block 1)
  // The midstate is the internal state AFTER processing Block 1.
  // Only the nonce (bytes 76-79) is in Block 2, so we can
  // pre-compute the midstate and only re-hash Block 2 per nonce!
```

This optimization (midstate caching) is WHY Bitcoin mining is fast. The PS computes the midstate ONCE per job, then the FPGA only re-hashes Block 2 (which changes with every nonce).

10. Step 6: Feed to PL and Start Mining

Once the PS has the midstate and block header, it writes everything to the PL's CSR registers and starts the engines.

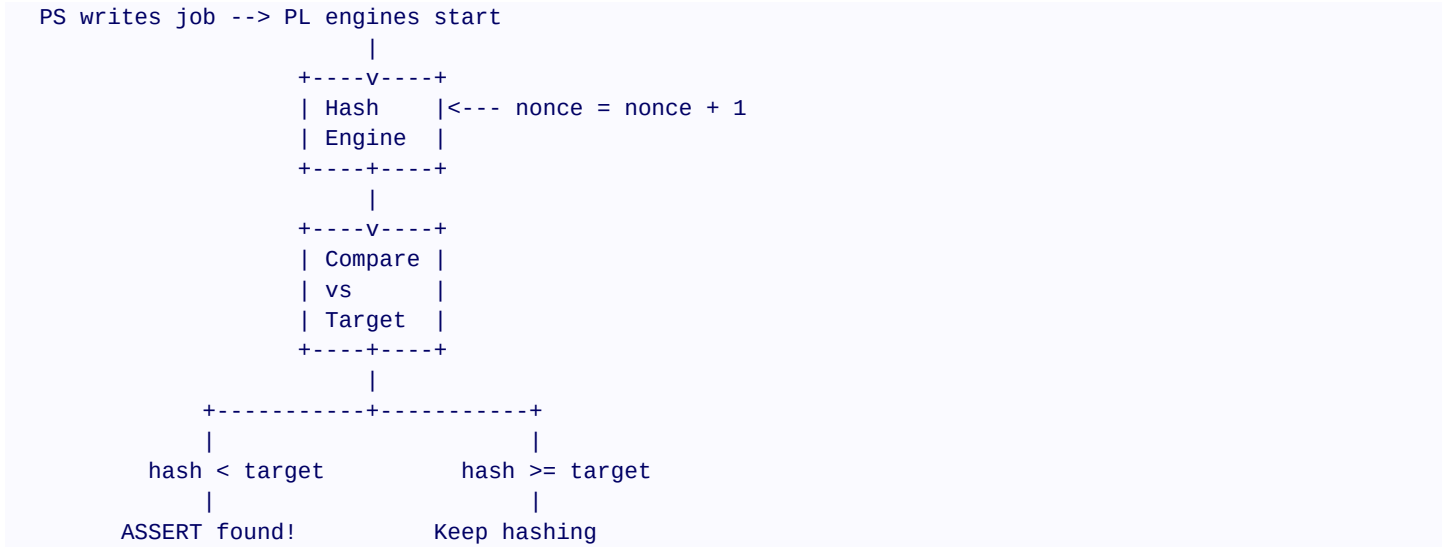
What the PS Writes

```
// In send_job_to_pl(ctx):

  // 1. Write 256-bit difficulty target (8 x 32-bit words)
  for (i = 0; i < 8; i++)
      pdsa_csr_write(hal, CSR_TARGET_BASE + i*4, target[i]);

  // 2. Write 256-bit midstate (8 x 32-bit words)
  for (i = 0; i < 8; i++)
      pdsa_csr_write(hal, CSR_MIDSTATE_BASE + i*4, midstate[i]);

  // 3. Write 640-bit job data (20 x 32-bit words)
  for (i = 0; i < 20; i++)
      pdsa_csr_write(hal, CSR_JOBDATA_BASE + i*4, job_data[i]);

  // 4. Write starting nonce
  pdsa_csr_write(hal, CSR_NONCE, ctx->nonce);

  // 5. Start mining!
  pdsa_csr_write(hal, CSR_CTRL, CTRL_START);
```

What Happens in the PL

- The axi_lite_csr stores all values in registers.
- The static shell routes them to the RP (rm_bitcoin).
- The SHA-256d engines start hashing: one engine does even nonces, one does odd nonces.
- After 132 cycles (1.32 us) of pipeline fill, the PL produces 2 hashes per cycle (125 MH/s at 100 MHz with 2 engines).
- Each hash is compared against the target. If hash < target: FOUND!

Mining Loop in PL



```
        Interrupt PS            (next nonce)
```

11. Step 7: Submit Share ("I found one!")

When the PL finds a hash below the target, it asserts the 'found' signal and triggers an interrupt (GIC SPI 89). The PS reads the result and submits it to the pool.

What the PS Does on Interrupt

```
1. Read CSR_STATUS to confirm STATUS_FOUND is set
  2. Read CSR_GOLDEN_NONCE (0x0B8) - the winning nonce
  3. Read CSR_RESULT_HASH (0x0C8-0xE4) - 8 words of the hash
  4. Build the full block header with the found nonce
  5. Submit to pool via stratum_submit_share()
```

The Submit Message

```
MINER sends: {"id":3,"method":"mining.submit",
    "params":[
      "worker1",         <- worker name
      "4",               <- job_id (from mining.notify)
      "00000000",         <- extranonce2
      "61cfa8e4",         <- ntime (timestamp used)
      "4a7b9c3d"          <- nonce your FPGA found!
    ]
  }
```

The Pool's Response

```
POOL responds: {"id":3,"result":true,"error":null}
  // TRUE = Accepted! Your share counted!
  // FALSE = Rejected. Check your job_id, ntime, nonce.
```

What is a Share?

A share is a hash that meets the POOL's target (easier than the Bitcoin network target). Think of it as proof that you did work. The pool counts shares from all miners. Even if your hash doesn't meet the Bitcoin network target, it still proves you were trying, and the pool rewards you for it.

Share vs Block

```
Bitcoin Network Target (very hard):
  0x00000000 00000000 00000000 00000000
  00000000 00000000 00000000 00000000FFFF...
  (LOTS of leading zeros)

  Pool Target (easier):
  0x00000000 00000000 00000000 FFFFFFFF...
  (fewer leading zeros)

  Your Found Hash:
  0x00000000 00000000 00000001 4A7B9C3D...

  Does it meet pool target? YES  --> Submit as SHARE
  Does it meet network target? NO --> Not a block, but pool
                                      still gives you credit
```

12. Step 8: Repeat (Keep Mining)

Mining is a continuous loop:

```
WHILE (connected to pool) {
      // Wait for pool to send a new job
      new_job = recv_stratum_message();
      if (new_job && new_job.clean_jobs) {
          // Abandon old work, start fresh
          stop_pl_engines();
          header = parse_job(new_job);
          midstate = compute_midstate(header);
          send_job_to_pl(midstate, header, target);
          start_pl_engines();
      }

      // While mining, check for found nonces
      if (pl_interrupt_happened()) {
          nonce = read_csr(CSR_GOLDEN_NONCE);
          submit_share(job_id, ntime, nonce);
      }

      // Every 5 seconds, evaluate PDSA
      if (timer_expired(5_seconds)) {
          pdsa_evaluate();
      }

      // Every second, update CSV log
      log_csv(hashrate, shares, state);
  }
```

This loop runs CONTINUOUSLY for hours or days. The pool sends new jobs every time there's a new block on the Bitcoin network (roughly every 10 minutes), but many pools send new jobs every 30 seconds with updated transactions.

What Happens When a Block is Found?

When ANY miner in the pool finds a block:

- The pool broadcasts the block to the Bitcoin network.
- The network verifies it (all nodes check the hash).
- The winning miner gets the coinbase reward + fees.
- The pool distributes shares to ALL miners proportionally.
- A new block starts, and the pool sends a new mining.notify with:

- New prev_hash (the block that was just found)

- clean_jobs = TRUE (abandon all old jobs)

- New coinbase, merkle_branch, etc.

13. Real JSON Messages (Copy-Paste Examples)

Here are REAL Stratum messages you would see if you connected to a pool. You can test this manually with netcat if you want!

Complete Session

```
===== CONNECT =====
  TCP: Connecting to btc.f2pool.com:3333...

  ===== SUBSCRIBE =====
  MINER> {"id":1,"method":"mining.subscribe",
           "params":["PDSA-FPGA/1.0"]}

  POOL>  {"id":1,"result":[
           [["mining.notify","ae6812eb4cd9"],
            ["mining.set_difficulty","fbb345e9"]],
           "08000002",4
          ],"error":null}

  ===== AUTHORIZE =====
  MINER> {"id":2,"method":"mining.authorize",
           "params":["bc1qabc123def456.rig1","x"]}

  POOL>  {"id":2,"result":true,"error":null}

  ===== NEW JOB =====
  POOL>  {"method":"mining.notify",
           "params":["4a","000000000000045a...",
            "0100000001000000...","ffffffff...",
            ["5c3c2d3e...","6a7b8c9d...",
             "1a2b3c4d...","9e8f7a6b..."],
            "20000000","1a2b3c4d","5e8f7a6b",true]}

  ===== SHARE ACCEPTED =====
  MINER> {"params":["bc1qabc123def456.rig1",
            "4a","00000000","5e8f7a6b","1a2b3c4d"],
            "id":3,"method":"mining.submit"}

  POOL>  {"id":3,"result":true,"error":null}
```

14. Where in the Code?

In the current project, all Stratum-related code has been removed. The mining loop uses create_simulated_job() instead of a pool connection. These files no longer exist in the source tree:

```
[REMOVED] ps_software/stratum/
  ==============================

  stratum_client.h (35 lines) -- DELETED
    - struct stratum_ctx_t (sock, job_id, etc.)
    - function declarations

  stratum_client.c (420 lines) -- DELETED
    - stratum_connect(), stratum_subscribe()
    - stratum_authorize(), stratum_get_job()
    - stratum_submit_share(), parse_stratum_message()

  [REMOVED] ps_software/mining/
  ============================

  block_header_parser.c (120 lines) -- DELETED
    - build_block_header(), compute_midstate()

  [CURRENT] ps_software/pdsa_main.c
  =================================

    - send_job_to_pl():
        Writes midstate, job_data, target to CSR

    - create_simulated_job():
        Creates dummy header + easy target (replaces pool job)

    - main loop:
        No pool polling, no share submission
        Every 5s: simulated PT decline -> PDSA eval -> DPR switch
```

15. What You Need to Change for Your Pool

The current project version runs in standalone mode with no pool connection. To add pool support back, you would need to restore the Stratum client and modify pdsa_main.c to use pool jobs instead of simulated jobs.

What Would Need to Change

```
1. RESTORE DELETED FILES:
     - stratum/stratum_client.c + stratum_client.h
     - mining/block_header_parser.c

  2. EDIT pdsa_main.c:
     - Add #include "../stratum/stratum_client.h"
     - Add POOL_HOST, POOL_PORT, POOL_WORKER defines
     - Add stratum_connect/subscribe/authorize after self-tests
     - Replace create_simulated_job() with parse_mining_job()
     - Add stratum_submit() in nonce-found handler
     - Add pool retry/reconnect logic

  3. UPDATE Makefile:
     - Add stratum_client.o and block_header_parser.o back to SRCS2
     - Add -I./stratum to INCLUDES
```

Current Project Status

The project now provides:

- Self-tests: NIST SW/PL, NIST KAT, Genesis block, PDSA+DPR
- Local mining: Simulated jobs with easy targets, real PL hashing
- PDSA evaluation: Simulated PT decline triggering DPR switches
- Benchmarking: Hashrate, nonce, DPR latency logged to CSV
- No network dependency: Runs entirely offline on the KV260

-- End of Stratum Protocol Explanation --